

LAPORAN PRAKTIKUM W5: ABSTRACT CLASS DAN INTERFACE

Diajukan untuk memenuhi salah satu tugas praktikum mata kuliah Pemrograman Berorientasi Objek.



Disusun Oleh:
Rainissa Azizah (251511059)
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026

Tugas 1: Abstract Class Animal

Pada tugas ini dibuat abstract class Animal dengan atribut nama, asal, dan jumlahKaki. Atribut tersebut diisi lewat constructor dan dapat diakses melalui method getter. Animal memiliki dua method, yaitu toShout() yang merupakan abstract method dan toEat() satu method biasa yang mencetak teks "like to eat". Karena memiliki abstract method, Animal tidak dapat diinstansiasi dan hanya berfungsi sebagai kelas induk. Berikut adalah kode dari abstract class Animal:

```
public abstract class Animal {
    private String nama;
    private String asal;
    private int jumlahKaki;

    public Animal(String nama, String asal, int jumlahKaki){
        this.nama = nama;
        this.asal = asal;
        this.jumlahKaki = jumlahKaki;
    }
    public String getNama() {
        return nama;
    }
    public String getAsal(){
        return asal;
    }
    public int getJumlahKaki(){
        return jumlahKaki;
    }
    abstract public void toShout();

    public void toEat(){
        System.out.println("like to eat");
    }
}
```

Kelas Dog, Chicken, dan Lion dibuat sebagai turunan dari Animal. Ketiganya memanggil constructor induk dengan super() untuk mengisi atribut, lalu mengimplementasikan toShout() sesuai suara masing-masing hewan. Method toEat() tidak ditulis ulang karena sudah tersedia di Animal dan diwarisi oleh ketiga kelas. Berikut adalah kode-kodenya:

A. Kode Class Dog

```
public class Dog extends Animal {  
    public Dog(String nama, String asal, int jumlahKaki){  
        super(nama, asal, jumlahKaki);  
    }  
    public void toShout(){  
        System.out.println("Guk guk!");  
    }  
}
```

B. Kode Class Chicken

```
public class Chicken extends Animal {  
    public Chicken(String nama, String asal, int jumlahKaki){  
        super(nama, asal, jumlahKaki);  
    }  
    public void toShout(){  
        System.out.println("Kukuruyuk!");  
    }  
}
```

C. Kode Class Lion

```
public class Lion extends Animal{  
    public Lion(String nama, String asal, int jumlahKaki){  
        super(nama, asal, jumlahKaki);  
    }  
    public void toShout(){  
        System.out.println("Rawr!");  
    }  
}
```

Pengujian dilakukan pada kelas TestAnimal. Ketiga objek disimpan dalam satu array bertipe Animal, kemudian ditampilkan satu per satu menggunakan looping. Data nama, asal, dan jumlah kaki diambil lewat getter, sedangkan suara dan cara makan ditampilkan dengan memanggil toShout() dan toEat(). Berikut adalah kode beserta hasil outputnya:

A. Kode TestAnimal

```
public class TestAnimal {  
    public static void main(String[] args) {  
        Animal[] daftar = {
```

```

        new Dog("Nanung", "Indonesia", 4),
        new Chicken("Wilson", "Spanyol", 2),
        new Lion("Simba", "Afrika", 4)
    };
    System.out.println("==== DAFTAR HEWAN =====");
    System.out.println();
    int no = 1;
    for (Animal a : daftar) {
        System.out.println(no + ". " + a.getClass().getSimpleName());
        System.out.println("-----");
        System.out.println("Nama   : " + a.getNama());
        System.out.println("Asal   : " + a.getAsal());
        System.out.println("Kaki   : " + a.getJumlahKaki());
        System.out.print("Suara : ");
        a.toShout();
        System.out.print("Makan : ");
        a.toEat();
        System.out.println();
        no++;
    }
}

```

B. Hasil output TestAnimal

```

==== DAFTAR HEWAN =====

1. Dog
-----
Nama   : Nanung
Asal   : Indonesia
Kaki   : 4
Suara : Guk guk!
Makan : like to eat

2. Chicken
-----
Nama   : Wilson
Asal   : Spanyol
Kaki   : 2
Suara : Kukuruyuk!
Makan : like to eat

3. Lion
-----
Nama   : Simba
Asal   : Afrika
Kaki   : 4
Suara : Rawr!
Makan : like to eat

```

Tugas 2: Goods dan Interface Taxable

Pada tugas ini dibuat kelas Goods dengan atribut description dan price. Atribut dibuat private dan diakses melalui getter agar prinsip encapsulation terpenuhi. Kelas Food, Toy, dan Book dibuat sebagai turunan dari Goods dengan tambahan atribut masing-masing, yaitu calories pada Food, minimumAge pada Toy, dan author pada Book.

A. Kode Class Goods

```
public class Goods {
    private String description;
    private double price;

    public Goods(String description, double price){
        this.description = description;
        this.price = price;
    }
    public String getDescription(){
        return description;
    }
    public double getPrice(){
        return price;
    }
    public void setDescription(String description){
        this.description = description;
    }
    public void setPrice(double price){
        this.price = price;
    }
    public void display(){
        System.out.println("Deskripsi: " + description);
        System.out.println("Harga: " + price);
    }
}
```

B. Kode Class Food

```
public class Food extends Goods {
    private int calories;

    public Food(String description, double price, int calories){
```

```

        super(description, price);
        this.calories = calories;
    }
    public int getCalories(){
        return calories;
    }
    public void setCalories(int calories){
        this.calories = calories;
    }
    public void display(){
        System.out.println("Kalori: " + calories);
    }
}

```

C. Kode Class Toy

```

public class Toy extends Goods implements Taxable {
    private int minimumAge;

    public Toy(String description, double price, int minimumAge){
        super(description, price);
        this.minimumAge = minimumAge;
    }
    public int getMinimumAge(){
        return minimumAge;
    }
    public void setMinimumAge(int minimumAge){
        this.minimumAge = minimumAge;
    }
    public double calculateTax(){
        return getPrice() * taxRate;
    }
    public void display(){
        System.out.println("Usia minimal: " + minimumAge);
        System.out.println("Harga Pajak: " + calculateTax());
    }
}

```

D. Kode Class Book

```

public class Book extends Goods implements Taxable{

```

```

    private String author;

    public Book(String description, double price, String author){
        super(description, price);
        this.author = author;
    }
    public String getAuthor(){
        return author;
    }
    public void setAuthor(String author){
        this.author = author;
    }
    public double calculateTax(){
        return getPrice() * taxRate;
    }
    public void display(){
        System.out.println("Author: " + author);
        System.out.println("Harga Pajak: " + calculateTax());
    }
}

```

Interface Taxable dibuat dengan atribut taxRate bernilai 0.06 dan method calculateTax(). Interface ini hanya diimplementasikan oleh Toy dan Book, sehingga Food tidak dikenai pajak sesuai diagram pada soal. Pada kedua kelas tersebut, calculateTax() menghitung pajak dari harga barang dikali taxRate. Berikut adalah kode dari interface Taxable:

```

public interface Taxable {
    double taxRate = 0.06;
    double calculateTax();
}

```

Pengujian dilakukan di class TestTaxable. Objek Food, Toy, dan Book dimasukkan ke dalam satu array bertipe Goods, lalu ditampilkan satu per satu dengan looping. Data yang sama untuk semua barang, seperti deskripsi dan harga, diambil lewat getter dari Goods. Data khusus tiap barang, seperti *calories*, *minimum age*, dan *author*, ditampilkan setelah jenis objeknya dicek dengan instanceof. Pengecekan instanceof Taxable juga dipakai untuk mengetahui barang mana yang kena pajak, sehingga pajak hanya muncul untuk Toy dan Book. Berikut adalah kode TestTaxable dan outputnya:

A. Kode TestTaxable

```
public class TestTaxable {
    public static void main(String[] args) {
        Goods[] daftar = {
            new Food("Roti", 15.000, 250),
            new Toy("Lego", 45.000, 10),
            new Book("Malice", 99.000, "Keigo Higashino")
        };
        System.out.println("==== DAFTAR BARANG ====");
        System.out.println();

        int no = 1;
        for (Goods g : daftar) {
            System.out.println(no + ". " + g.getClass().getSimpleName());
            System.out.println("-----");
            System.out.println("Deskripsi : " + g.getDescription());
            System.out.println("Harga      : Rp" + g.getPrice());

            if (g instanceof Food) {
                Food f = (Food) g;
                System.out.println("Kalori      : " + f.getCalories());
            } else if (g instanceof Toy) {
                Toy t = (Toy) g;
                System.out.println("Usia min. : " + t.getMinimumAge());
            } else if (g instanceof Book) {
                Book b = (Book) g;
                System.out.println("Penulis   : " + b.getAuthor());
            }

            if (g instanceof Taxable) {
                Taxable tx = (Taxable) g;
                System.out.println("Pajak     : Rp" + tx.calculateTax());
            } else {
                System.out.println("Pajak     : -");
            }
            System.out.println();
            no++;
        }
    }
}
```

```
}  
}
```

B. Hasil output

```
===== DAFTAR BARANG =====  
  
1. Food  
-----  
Deskripsi : Roti  
Harga      : Rp15000.0  
Kalori     : 250  
Pajak      : -  
  
2. Toy  
-----  
Deskripsi : Lego  
Harga      : Rp45000.0  
Usia min.  : 10  
Pajak      : Rp2700.0  
  
3. Book  
-----  
Deskripsi : Malice  
Harga      : Rp99000.0  
Penulis    : Keigo Higashino  
Pajak      : Rp5940.0
```

Lesson Learn

Dari kedua tugas ini, dipelajari bahwa abstract class digunakan untuk kelas induk yang tidak perlu dibuat objeknya tetapi menyimpan atribut dan perilaku yang sama untuk semua turunannya. Abstract method memaksa setiap turunan untuk menulis perilakunya sendiri, seperti `toShout()` pada hewan. Interface dipelajari sebagai kontrak yang bisa diterapkan pada kelas tertentu saja. Hal ini terlihat pada `Taxable` yang hanya diterapkan pada `Toy` dan `Book` walaupun ketiganya berasal dari induk yang sama.

Pada class `Goods`, objek `Food`, `Toy`, dan `Book` dimasukkan ke dalam satu array bertipe `Goods`. Karena Java menganggap semuanya hanya `Goods`, maka method khusus seperti `getCalories()`, `getMinimumAge()`, dan `getAuthor()` tidak bisa langsung dipanggil. Untuk mengatasi hal tersebut, digunakan `instanceof`, yaitu cara untuk mengecek objek itu sebenarnya `Food`, `Toy`, atau `Book`. Setelah dicek, objek diubah ke jenis aslinya supaya method khususnya bisa dipakai. `Instanceof` juga dipakai untuk mengecek apakah barang itu implement `Taxable`, jadi pajak hanya muncul untuk barang yang memang kena pajak.

Link Repo

<https://github.com/rainissa/PBO-Praktik.git>